

"""

UMAExecutor – schedules and runs a UMA_IR graph against a UMAClient kernel.

This is the bridge layer:

```
SemanticEngine (compiler/planner)
    ↓ emits
UMA_IR
    ↓ consumed by
UMAExecutor          ← THIS FILE
    ↓ drives
UMAClient (kernel)
```

The executor knows nothing about semantic intent. It only:

1. Iterates the IR schedule
2. Dispatches each node to the correct kernel call
3. Records friction at every checkpoint node
4. Returns a structured result

No semantic logic leaks into the kernel.

"""

```
from __future__ import annotations
import numpy as np
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional

from uma.core.state import FieldPosterior
from uma.semantic.ir import UMA_IR, IRNode
from uma.semantic.friction import SemanticFriction, FrictionRecord
from uma.semantic.constraints import ConstraintSet
from uma.semantic.inarticulation import (
    Inarticulator, NullInarticulator, ProductionMetrics
)

@dataclass
class ExecutorResult:
    """Raw output from UMAExecutor.run()."""
    is_closed: bool
    closure_node: Optional[str] # node_id where closure was declared
    friction_records: List[FrictionRecord]
    friction_summary: Dict[str, Any]
    final_posterior: FieldPosterior
    z_final: np.ndarray
```

```
production_metrics: ProductionMetrics
nodes_executed: int
```

```
class UMAExecutor:
```

```
    """
```

```
    Executes a UMA_IR schedule against a live UMAClient.
```

```
    Stateless across runs – all state lives in the client and friction tracker.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        friction: SemanticFriction,
```

```
        inarticulator: Optional[Inarticulator] = None,
```

```
        stop_on_closure: bool = True,
```

```
        verbose: bool = False,
```

```
    ):
```

```
        self.friction = friction
```

```
        self.inarticulator = inarticulator or NullInarticulator()
```

```
        self.stop_on_closure = stop_on_closure
```

```
        self.verbose = verbose
```

```
    def _log(self, msg: str) -> None:
```

```
        if self.verbose:
```

```
            print(f"[UMAExecutor] {msg}")
```

```
    # -----
```

```
    def run(self, ir: UMA_IR, client) -> ExecutorResult:
```

```
        """
```

```
        Execute ir.schedule against client.
```

```
        client must already be initialized (client.posterior_state is not None).
```

```
        """
```

```
        if client.posterior_state is None:
```

```
            raise RuntimeError("UMAClient must be initialized before execution.")
```

```
        posterior = client.posterior_state
```

```
        self.friction.reset(posterior)
```

```
        z_target = ir.metadata.get("z_target", np.zeros(posterior.dim))
```

```
        node_map = {n.node_id: n for n in ir.nodes}
```

```
        last_constraint_report: Dict[str, Any] = {
```

```
            "converged": True, "final_total_violation": 0.0
```

```
        }
```

```
        closure_node: Optional[str] = None
```

```

nodes_executed = 0
t = client.t

for node_id in ir.schedule:
    node: IRNode = node_map[node_id]
    nodes_executed += 1

    # — EVOLVE —————
    if node.kind == "evolve":
        dt = node.payload.get("dt", 0.04)
        try:
            psi = client.projection.lift(posterior.mean)
            psi = client.dynamics.step(psi, dt, rng=client.rng)
            z_new = client.projection.project(psi)
            # Rebuild posterior with updated mean, preserve  $\Sigma$ 
            posterior = _shift_mean(posterior, z_new)
        except Exception as e:
            self._log(f"evolve error at {node_id}: {e}")
            t += dt

    # — OBSERVE —————
    elif node.kind == "observe":
        obs = node.payload["observation"]
        y = node.payload["y"]
        try:
            posterior, _ = client.filter.update(posterior, obs, y)
        except Exception as e:
            self._log(f"observe error at {node_id}: {e}")

    # — CONSTRAIN —————
    elif node.kind == "constrain":
        cs: ConstraintSet = node.payload["constraint_set"]
        z = posterior.mean.copy()
        z_anchored, report = cs.apply(z)
        last_constraint_report = report
        posterior = _shift_mean(posterior, z_anchored)

    # — CHECKPOINT —————
    elif node.kind == "checkpoint":
        rec = self.friction.update(posterior, t)
        self._log(
            f"{node_id}  t={t:.3f}  "
            f"E={rec.E:.4f}  dz/dt={rec.dz_dt:.4e}  "
            f"friction={rec.friction:.4f}  closed={rec.closed}"
        )
        if self.stop_on_closure and rec.closed:
            closure_node = node_id

```

```

        self._log(f"Closure ( $\Omega$ ) at {node_id}")
        break

    # Update client state
    client.posterior_state = posterior
    client.t = t

    z_final = posterior.mean.copy()
    friction_summary = self.friction.summary()

    production_metrics = self.inarticulator.translate(
        z_omega=z_final,
        posterior=posterior,
        z_target=z_target,
        friction_summary=friction_summary,
        constraint_report=last_constraint_report,
    )

    return ExecutorResult(
        is_closed=self.friction.is_closed,
        closure_node=closure_node,
        friction_records=self.friction.records,
        friction_summary=friction_summary,
        final_posterior=posterior,
        z_final=z_final,
        production_metrics=production_metrics,
        nodes_executed=nodes_executed,
    )

# -----
# Helper - shift posterior mean without touching covariance
# -----

def _shift_mean(posterior: FieldPosterior, z_new: np.ndarray) -> FieldPosterior:
    """Rebuild posterior with updated mean; covariance structure unchanged."""
    if posterior.representation == "full":
        return FieldPosterior.full(z_new, posterior.Sigma)
    elif posterior.representation == "lowrank":
        return FieldPosterior.lowrank(z_new, posterior.L, posterior.diag_floor)
    elif posterior.representation == "ensemble":
        delta = z_new - posterior.mean
        return FieldPosterior.ensemble(posterior.ensemble + delta)
    elif posterior.representation == "particle":
        delta = z_new - posterior.mean
        return FieldPosterior.particle(
            posterior.particles + delta, posterior.log_weights

```

```
)  
return posterior
```